

HW04 – Automation Testing Report

1. Student Information

Field	Value
Student name	NGUYEN THIEN NHA TRAN
Student ID	23127272
Class / Cohort	23KTPM2
Assignment	HW04-AI – Automation Testing
AI tool	OpenAI Codex

2. Scope and Feature Selection

The same three web features selected in HW02 were automated:

Pool	Feature	HW02 source	Automated cases
A	FR-03 Forgot password and password reset	../HW02-AI/main report/FR-03-domain-testing.md	12
B	FR-11 User order history	../HW02-AI/main report/FR-11-domain-testing.md	12
C	FR-14 Category management	../HW02-AI/main report/FR-14-domain-testing.md	12

3. Automation Design

The suite uses Playwright with external JSON data. Each JSON row contains a stable ID, title, action key, inputs, expected result, and requirement IDs. The `.spec.js` files iterate over those rows; variable test inputs are not stored in inline arrays.

The design is hybrid:

- UI pages and React behavior are always the real SUT.
- Main navigation and authentication gates are exercised in the UI.
- Deterministic network fixtures provide rare order states and disposable category state without modifying the SUT database.
- Fixtures set up preconditions only; requirement assertions evaluate the real frontend behavior.

Assertion patterns include URL, visibility, text/content, count/structure, attributes, values, classes, and negative assertions.

4. Browser and Report Configuration

The locally available browser projects are:

- Playwright Chromium

- Installed Google Chrome
- Installed Microsoft Edge

Each feature was invoked independently on every browser. The nine report directories are under `automation/reports/<feature>/<browser>/`. Every report contains:

- visible title `Run by: 23127272;`
- an ISO run timestamp;
- feature and browser metadata;
- test annotations, screenshots, videos, and traces for failures.

`scripts/validate-reports.ps1` decodes the embedded Playwright `report.json` and verifies these fields instead of trusting directory names.

5. Execution Summary

Feature	Automated	Browser runs	Executions	Passed	Failed	Flaky	Skipped
FR-03	12	3	36	6	30	0	0
FR-11	12	3	36	33	3	0	0
FR-14	12	3	36	30	6	0	0
Total	36	9	108	69	39	0	0

The result is deterministic across the three browsers: every case has the same outcome on each browser. Failures were consolidated into nine unique defects in `bug-report/bug-report.md`.

6. AI-generated Script Review and Human Fixes

The first generated matrix runner attempted to spawn `npm.cmd`. On this Windows environment, the child process did not start and the runner misleadingly summarized all nine runs as failed in about one second. Human review identified the absence of Playwright output and report files. The runner was corrected to call `process.execPath` with `require.resolve('@playwright/test/cli')`, and it then executed all 108 browser tests.

The initial OTP assertion used word boundaries. Rendered React text concatenated the OTP with the next element text, so a correct visible token did not match even though it was present. The assertion was corrected to numeric boundaries using non-digit checks.

The initial 7.5-second assertion timeout made known missing-element failures unnecessarily slow. It was reduced to three seconds after confirming that mocked UI states render in under one second.

The original browser plan assumed Firefox and WebKit runtimes. Installation was not authorized, so the environment was inspected and the executable projects were changed to Chromium, installed Chrome, and installed Edge. This limitation is explicit because all three use the Chromium engine.

Selectors were scoped to the target feature. For example, the back-to-login locator was restricted to `main`; otherwise the global header login link caused a false pass.

7. Failure Review

Before classifying failures, the test code, request fixture, rendered text, screenshot, and SUT source were compared with the published requirement. The 39 failed executions represent nine repeated product defects, not 39 distinct bugs. See [bug-report/bug-report.md](#).

8. Agent Skills

Two reusable task skills were created instead of one assignment-specific skill:

- `convert-domain-cases-to-playwright`: converts existing requirement/domain cases into external-data Playwright tests and enforces human-review gates.
- `run-playwright-browser-matrix`: executes isolated feature-browser runs and defines the attributable report contract.

Both skill directories passed the official `quick_validate.py`. The first skill's data validator was executed against all three feature data files.

9. Limitations and Remaining Student Actions

- Chrome, Edge, and Chromium are three browser executables but use one rendering engine. Firefox/WebKit remains the stronger compatibility matrix if their runtimes are installed later.
- Network fixtures make edge-state UI tests deterministic; they do not replace separate backend/API authorization testing.
- GitHub Issues must still be created from the nine reviewed bug rows and screenshots.
- The narrated Vietnamese video, `whoami/hostname` evidence, YouTube links, and PDF exports require the student.
- The assignment requires at least eight meaningful test-script commits over at least four real days. This cannot be fabricated after implementation.

10. Reproduction Commands

```
cd HW04-AI\automation
npm install
npm run validate:data
npm run test:matrix
npm run validate:reports
```